1. Conversational Buffer Memory

Core idea in one line

Keep every message in a list, and send the whole list to the model on every single call.

The analogy

Imagine talking to someone who forgets everything the instant you stop speaking. To have a conversation, you'd have to repeat the entire discussion from the beginning each time you say something new. That's exactly what buffer memory does — and the "someone" is the LLM.

Why does this exist?

Because **LLMs are stateless**.

Every API call is independent. The model has no idea what was said one second ago. It only knows what's inside the current request. If you don't pass the history yourself, the model has zero context.

So the memory isn't inside the model. **Memory is something you build outside the model and hand to it.** Buffer memory is the simplest possible version of that: a list.

```
Turn 1 → send: [system, user_1]
Turn 2 → send: [system, user_1, assistant_1, user_2]
Turn 3 → send: [system, user_1, assistant_1, user_2, assistant_2, user_3]
```

When do you use it?

- Short conversations — roughly under 15–20 turns
- Prototypes and demos where you're proving the idea works
- Any case where losing even one detail is unacceptable and the conversation is guaranteed to stay short.
- It's the baseline everything else is measured against

Advantages

- **Perfect recall.** Nothing is lost, nothing is distorted. Every word is exactly as it was said.
- **Dead simple.** It's an `append` to a list. No extra LLM calls, no database, no tuning.
- **Zero added latency.** No summarising step, no retrieval step.

- **Easy to debug.** What you see in the list is exactly what the model sees.

Disadvantages

- **Cost grows quadratically.** This is the killer. Turn 100 sends 100 turns' worth of tokens. And you pay on *every* turn. Total cost across a conversation grows roughly with the square of its length, not linearly.
- **It hits a hard wall.** Every model has a context window. Cross it and the call doesn't degrade — it throws an error. Your app breaks in production.
- **Latency creeps up.** Bigger prompt, slower response, every turn.
- **Noise dilutes signal.** By turn 60 the genuinely important fact from turn 3 is buried in 57 turns of small talk.

Worked example — FinCoach

A user opens FinCoach and says: *"I'm 29, earning ₹18 LPA, and I want to start a SIP."*

Turns 1–10 go beautifully. The agent remembers the age, the salary, the goal — perfect recall, because all of it is being re-sent every time.

By turn 60 — tax questions, insurance questions, a detour about home loans — the request being sent to the API is enormous. Cost per turn has multiplied. Latency is noticeable. And one more long turn tips it over the context limit, and FinCoach crashes mid-conversation.

In a RAG project: buffer memory is what you use for the chat history *around* the retrieval. The user asks a question, you retrieve documents, you answer. Turn two, the user says *"and what about the second point?"* — without buffer memory the agent has no idea what "the second point" refers to. Buffer memory holds the thread of the conversation; RAG supplies the facts. Two different jobs.

The important insight for freshers: **RAG is not memory.** RAG retrieves from documents. Memory retrieves from the conversation. Many people confuse the two.

Hybrid note

Buffer memory is almost never the long-term half of a hybrid — it has no compression and no persistence across sessions. But it *is* the short-term half in the simplest hybrid there is:

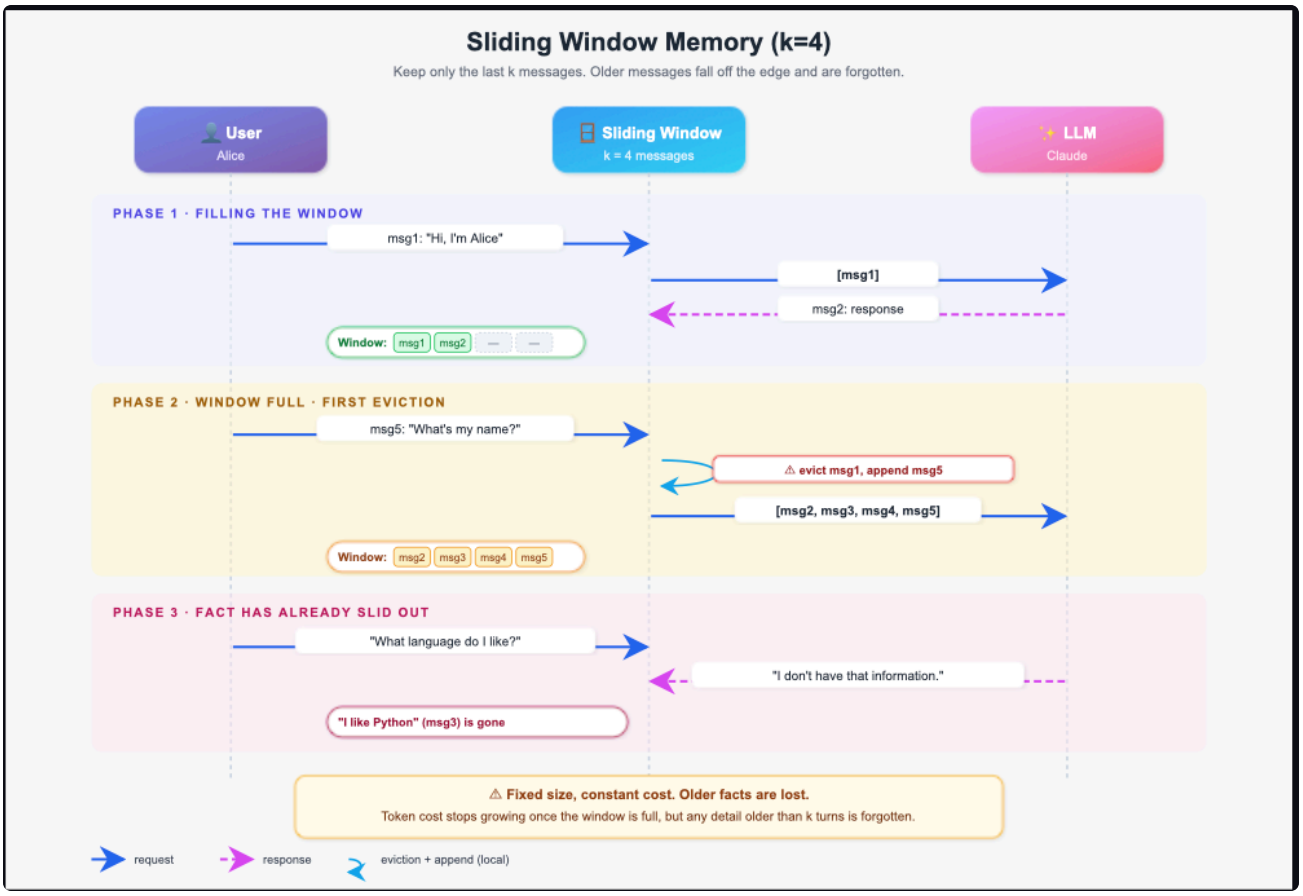
> **Buffer (short-term) + Vector Store (long-term).** Buffer holds the current session word-for-word so pronouns and follow-ups resolve correctly. The vector store holds everything from previous sessions and gets queried only when the current question needs history.

In practice, though, once you're building for production you'd swap the buffer for **token buffer** (type 5) — same job, but with a hard cost ceiling. We'll get there.

The one-line verdict

Correct, but does not scale. Use it to learn, use it for demos, replace it before production.

2 . Sliding Window Memory



Core idea in one line

Send only the **last N turns**. As new turns arrive, the oldest fall off the back.

The analogy

A spotlight moving along a long script. It only lights the last few pages. As new pages arrive, the spotlight moves forward and earlier pages fall into darkness.

```
Window size = 3

After Turn 3:  [T1] [T2] [T3]
After Turn 4:       [T2] [T3] [T4]     ← T1 falls off
After Turn 5:            [T3] [T4] [T5] ← T2 falls off
```

Why does this exist?

To fix buffer memory's cost problem directly. With a buffer, turn 100 sends 100 turns. With a window of 10, turn 100 sends exactly 10 turns — same as turn 10. **Cost stops compounding and becomes constant.**

One point that matters enormously in production: old messages are **evicted from the active window, not deleted.** In a toy implementation they are thrown away. In a real system they are archived to a database or vector store so they can be retrieved later. This distinction is the bridge to type 6.

When to use

- Chat where only the recent thread matters — support triage, casual Q&A
- High-volume, cost-sensitive products
- Any conversation where the topic naturally moves on and old turns become irrelevant
- As the short-term layer under a long-term store that holds the archive

Advantages

- **Constant, predictable cost.** Turn 500 costs the same as turn 10.
- **Never overflows the context window.** Structurally impossible.
- **No extra LLM calls.** No latency penalty at all.
- **Trivial to implement.** A `deque` with `maxlen` does it in one line.

Disadvantages

- **Silent amnesia.** The agent does not know it forgot. It will confidently ask a question you already answered.
- **Window size is measured in turns, not tokens.** Five turns might be 200 tokens or 5,000 depending on how verbose the conversation is — so the cost cap is loose.
- **Important early facts die with unimportant ones.** Turn 1 is treated exactly like turn 48.

Worked example — FinCoach

Window = 10. Chiru states his salary at turn 1. At turn 15 he asks *"how much should I put in a SIP?"* — the salary has fallen out of the window. FinCoach asks for it again. The user thinks the product is broken, because from their point of view, it is.

In a RAG project: this is the standard chat-history handling in nearly every RAG chatbot you will build. Keep the last 5–10 turns for follow-up resolution; the retrieved documents

supply the substance. If a user needs something from 30 turns ago, that is a retrieval problem, not a window problem.

Hybrid note

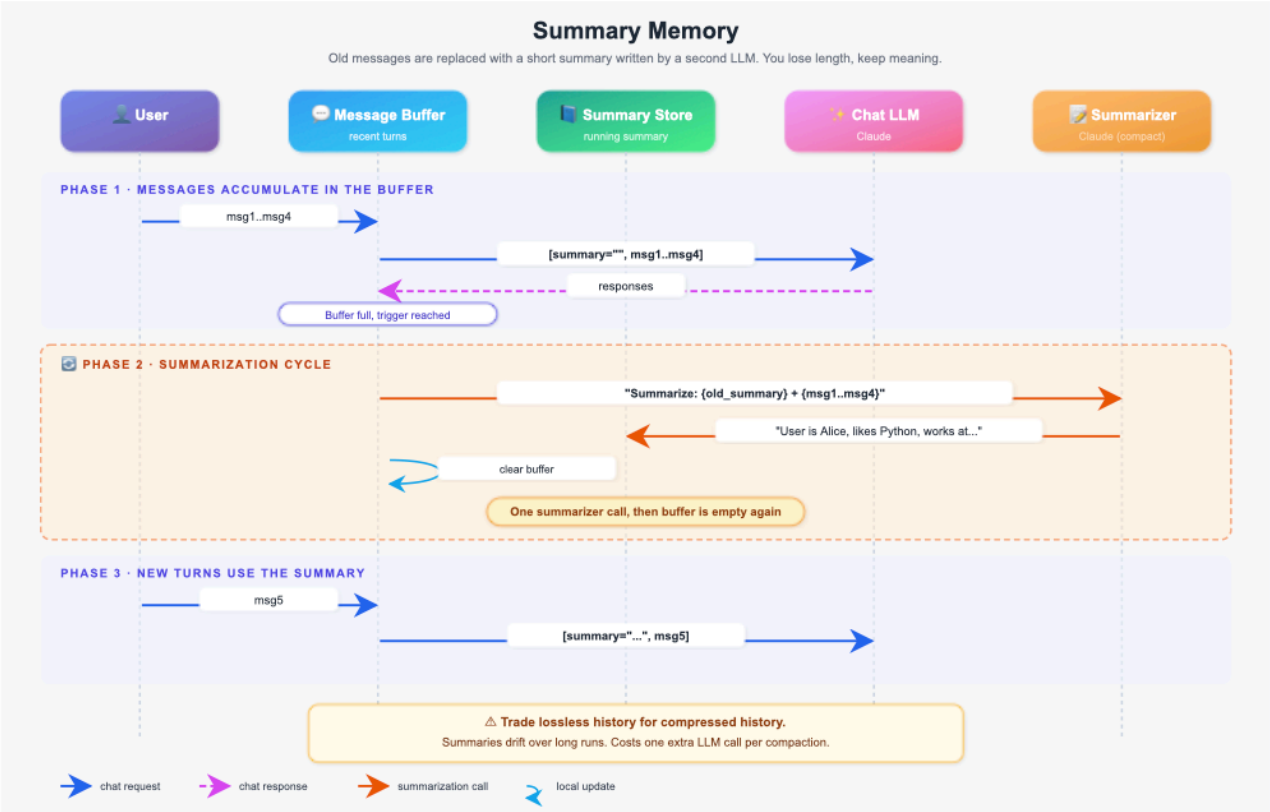
This is the **most common short-term half in production**. Pair it with:

- **Vector Store (6)** for the archive — evicted turns get embedded and stored, and are searched back when relevant. This combination is the backbone of most real memory systems.
- **Entity Memory (7)** so that stable facts like salary and risk profile live in a profile that is always injected, and never depend on being inside the window.

Verdict

Solves cost. Creates amnesia. Never ship it alone — always pair it with an archive.

3 . Summary Memory



Core idea in one line

When the conversation gets long, use the LLM itself to compress the oldest turns into a short summary, and put the summary in the context instead of the raw turns.

The analogy

A newspaper that rewrites yesterday's news into one paragraph each morning. The full detail of yesterday is gone but the meaning survives. Yesterday becomes a paragraph; last week becomes a sentence.

```
[System Prompt]
[SUMMARY: Chiru earns ₹1,20,000/month, expenses ₹60,000, has an FD of
 ₹50,000 maturing soon, is risk-averse, asked about SIPs and was
 advised to consider debt funds.]
[Turn 8 — verbatim]
[Turn 9 — verbatim]
[Turn 10 — verbatim]
```

Why does this exist?

Sliding window throws information away with a hard cutoff. Summary memory keeps the *meaning* while dropping the *wording*. The salary from turn 1 survives to turn 50 — not as the original sentence, but distilled into a paragraph.

Two mechanics to teach explicitly:

- The summary is written by a **second LLM call**, not by hand-coded rules. This is *abstractive* summarisation — the model writes new text capturing the essence.
- Summarisation fires on a **threshold** (every N turns, or when tokens cross a limit), not on every turn. Summarising every turn would be slow and expensive.

When to use

- Long advisory or coaching conversations where early context genuinely matters
- Sessions that run for an hour or more
- When you would rather have a fuzzy memory of turn 1 than no memory of turn 1

Advantages

- **Unbounded conversation length.** The context stays small no matter how long the session runs.
- **Early facts survive.** The core reason to pick this over a sliding window.
- **Big compression ratio.** Fifty turns can collapse into a paragraph.

Disadvantages

- **Lossy, and the loss compounds.** Each summarisation cycle can drop detail or shift emphasis. Summarise a summary a few times and you get **summary drift** — the agent's memory quietly diverges from what actually happened.

- **Exact numbers are the first casualty.** "₹1,20,000" can become "a good salary." In a financial product that is a serious failure.
- **Extra cost and latency.** Every summarisation is an additional API call, and the user waits for it.
- **Non-deterministic.** Run the same conversation twice and get two different summaries. Hard to debug.

Worked example — FinCoach

Chiru says his FD is ₹50,000 maturing in three months. After three summarisation cycles the summary reads *"has a small fixed deposit maturing shortly."* At turn 60 he asks *"how should I split the FD money?"* and FinCoach gives a vague answer because the exact number is gone.

Fix to teach here: write the summarisation prompt to *explicitly preserve* the fields that must never be lost — "preserve all numeric values exactly: salary, expenses, asset amounts, dates." Prompt design is what makes or breaks this technique.

In a RAG project: useful when a user works through a long document analysis session. Early turns established what they are looking for and why; the summary carries that intent forward so retrieval stays aligned to their actual goal.

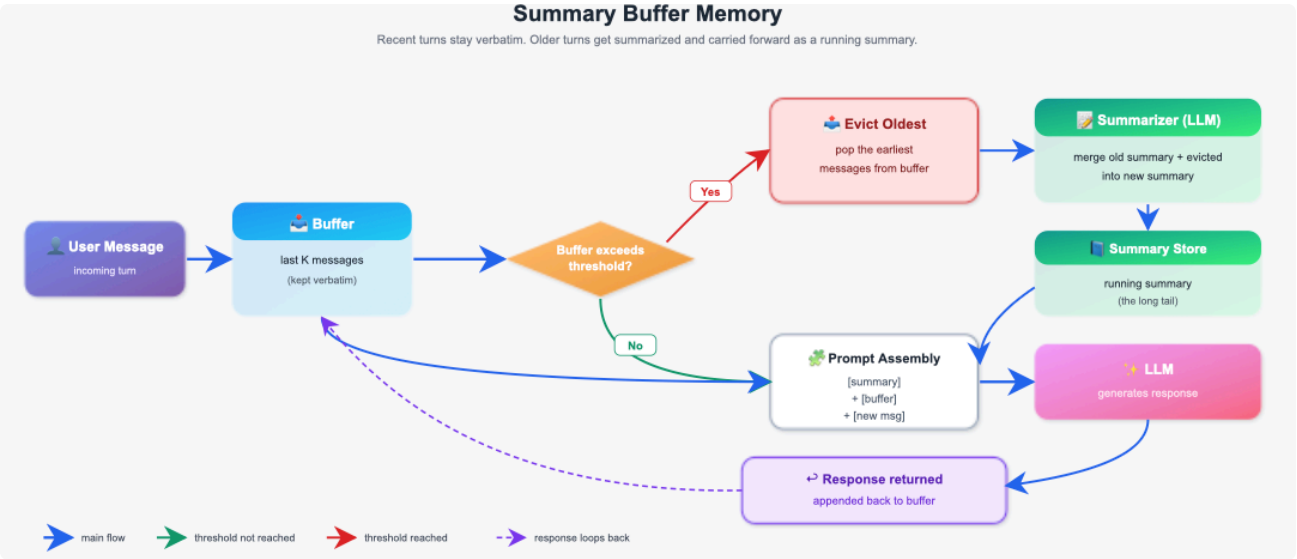
Hybrid note

Rarely used alone in production because raw summary memory keeps too few recent turns verbatim. It is almost always upgraded to type 4. Where it *is* used alone: batch or offline agents where latency does not matter.

Verdict

Preserves meaning, loses precision. Good idea, incomplete implementation — type 4 is the finished version.

4. Summary Buffer Memory



Core idea in one line

Keep recent messages word-for-word **and** compress older ones into a running summary — two zones in one system.

The analogy

A financial advisor who opens each meeting with a one-page briefing summarising previous sessions, then takes fresh notes during the current meeting. The briefing covers the past; the notepad covers the present.

```
┌──────────────────────────────────────────┐
│ [System: FinCoach persona]               │
│ [System: SUMMARY — compressed past]      │ ← permanence
│ [Turn N-2 verbatim]  ┐                   │
│ [Turn N-1 verbatim]  ├ token-capped recent zone │ ← recency
│ [Turn N   verbatim]  ┘                   │
└──────────────────────────────────────────┘
```

Why does this exist?

Because human memory works this way and so should the agent's. You recall the last few sentences of a phone call almost word-for-word, and the earlier part only as gist.

The upgrade over type 3 is that the recent zone is capped by **tokens, not turns**. You are billed in tokens, so tokens are the correct control unit. Five turns might cost 200 tokens or 5,000; a 2,000-token budget always costs at most 2,000.

Second upgrade: **overflow triggers compression, not rejection.** Type 1 raised an error and made the caller deal with it. Here, when the recent zone exceeds its budget, the oldest messages are automatically folded into the summary. The system self-heals.

When to use

- Long-running assistants: customer support, coaching, advisory
- Any product where a session can run for hours and cost must stay bounded
- **This is the default short-term choice for most production chat agents.**

Advantages

- **Best of both.** Precise recent context plus historical awareness.
- **Predictable cost with unbounded length.** The ceiling holds no matter how long the session runs.
- **Self-managing.** No caller-side overflow handling needed.
- **Recent turns are never distorted** — pronouns, corrections and follow-ups resolve correctly.

Disadvantages

- **Still lossy in the summary zone.** Drift is reduced, not eliminated.
- **Two things to tune, not one.** The transition threshold and the split between summary budget and buffer budget. Get the split wrong and you either drift too fast or pay too much.
- **Latency spikes on compression.** Most turns are fast; the turn where compression fires is noticeably slower.
- **Most complex short-term type to implement.**

Worked example — FinCoach

Chiru's forty-turn session: turns 1–34 are compressed into a paragraph holding salary, expenses, FD amount and risk profile. Turns 35–40 are verbatim. He asks *"actually, make it ₹8,000 instead"* — "it" refers to a SIP amount from turn 38, which is verbatim, so the correction lands correctly. Meanwhile the salary from turn 1 is still in the summary, so the advice stays personalised.

In a RAG project: the right choice for a document-analysis assistant. The summary holds what the user is investigating; the buffer holds the immediate back-and-forth about the current retrieved chunk.

Hybrid note

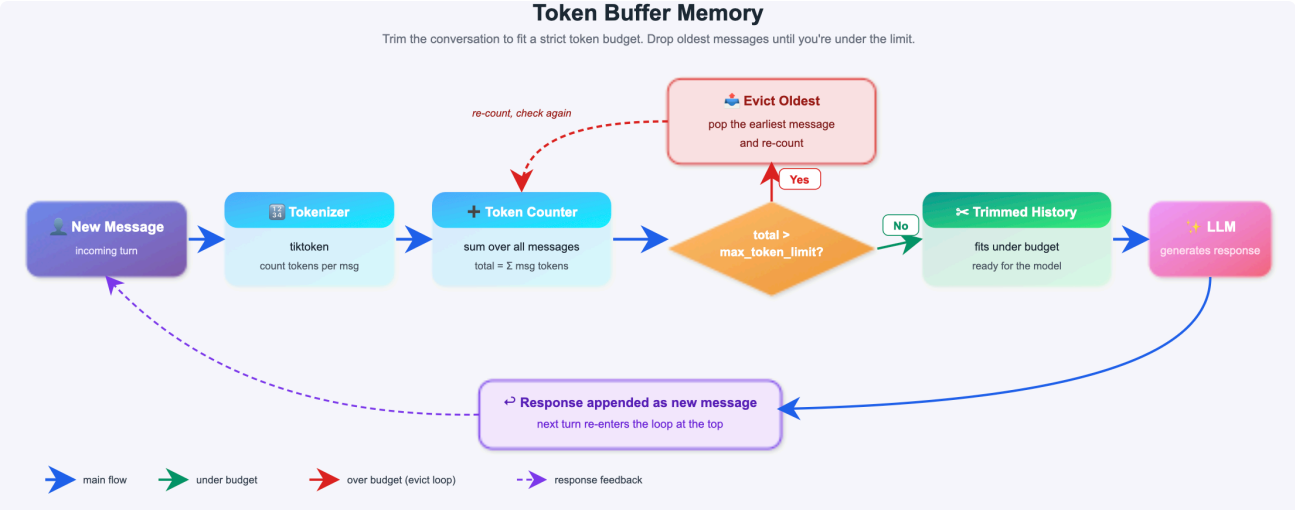
This *is* a hybrid — it fuses types 1 and 3. In a full architecture it is the short-term layer, paired with **Entity Memory (7)** for exact facts that must never drift, and **Vector Store (6)** or **Episodic (8)** for cross-session recall.

> **Design rule worth memorising:** put anything numeric or contractual in Entity Memory, and let the summary handle narrative context. Never trust a summary with a number.

Verdict

The strongest general-purpose short-term memory. If you pick one short-term type for a chat product, pick this.

5. Token Buffer Memory



Core idea in one line

Keep a buffer that **never exceeds a fixed token budget** — when a new message would breach it, drop the oldest messages one at a time until it fits.

The analogy

Packing a suitcase with a strict weight limit. You do not count items — you weigh them, and remove the oldest until everything fits.

```
Token budget = 500

Turn 4 arrives (65t) → total would be 540t ❌ over
  → drop T1-user (60t) → 480t, still over
  → drop T1-ai   (80t) → 400t, now fits ✅
  → add T4-user  (65t) → 465t final
```

Why does this exist?

Because sliding window drops by turn count, and message count is a poor proxy for cost. One message with a pasted code block or a table can cost thousands of tokens; a

message saying "yes" costs one. Token buffer replaces that blunt count with exact measurement using a real tokeniser.

This precision matters in production. An oversized prompt either truncates silently or errors out. An undersized prompt wastes expensive context. Token buffer eliminates both.

When to use

- **When latency is the top constraint.** No compression call means no latency spike, ever.
- When cost must be exactly, auditably predictable
- High-throughput systems serving thousands of concurrent users
- When you already have a long-term store handling recall, so dropping old messages is safe

Advantages

- **The most predictable cost model of any type.** `input = system_prompt + min(history, budget)`. That is the whole formula.
- **Zero added latency.** No summarisation, no retrieval, no extra API calls.
- **No external dependencies.** No vector DB, no second model.
- **Simplest production-grade option.** Easiest of all five to get right.

Disadvantages

- **Total information loss.** Evicted messages are gone from context with no compressed trace — worse than type 4 on recall.
- **Can break turn pairs.** It drops individual messages, so you can end up with an assistant reply whose user question was evicted. Confusing for the model. Production implementations offer an `evict_in_pairs` flag for exactly this reason.
- **Same silent amnesia as sliding window.**
- **Requires the right tokeniser.** Wrong tokeniser, wrong counts, wrong budget.

Worked example — FinCoach

Chiru pastes his entire portfolio statement — one message, 3,000 tokens. A sliding window of 10 turns would happily send all 3,000 and blow the cost model. Token buffer sees the budget breach immediately, evicts older messages, and the call stays inside its ceiling. Cost never surprises you.

In a RAG project: essential, because retrieved chunks are large and variable. You cannot budget chat history by turn count when a single turn might carry 4,000 tokens of retrieved

context. Token buffer is the only short-term type that handles this correctly.

Hybrid note

This is the answer to your specific question about pairing token buffer with a long-term type.

Token Buffer (short-term) + Vector Store (long-term) is the classic production pattern, and the split of responsibility is clean:

Layer	Holds	Costs	Answers
Token buffer	Last ~2,000 tokens verbatim	Fixed, capped	"what did we just say?"
Vector store	Every turn ever, embedded	Retrieval only when needed	"what did we discuss months ago?"

When do you use this pairing instead of Summary Buffer? Decide on latency:

- **Choose token buffer + vector store** when response time is the priority. Nothing blocks. Old turns are archived to the vector store asynchronously, in the background, so eviction costs the user nothing. Right for consumer apps, high-concurrency products, anything where p99 latency is a metric someone watches.
- **Choose summary buffer** when recall inside the session matters more than speed and you would rather pay a latency spike than lose the thread. Right for advisory, coaching, and long analytical sessions.

Also strong: **Token Buffer + Entity Memory (7)**. The buffer handles the conversation; the entity profile — a fixed, small block — is injected on every call so salary and risk profile are always present regardless of what got evicted. Cheap and highly effective.

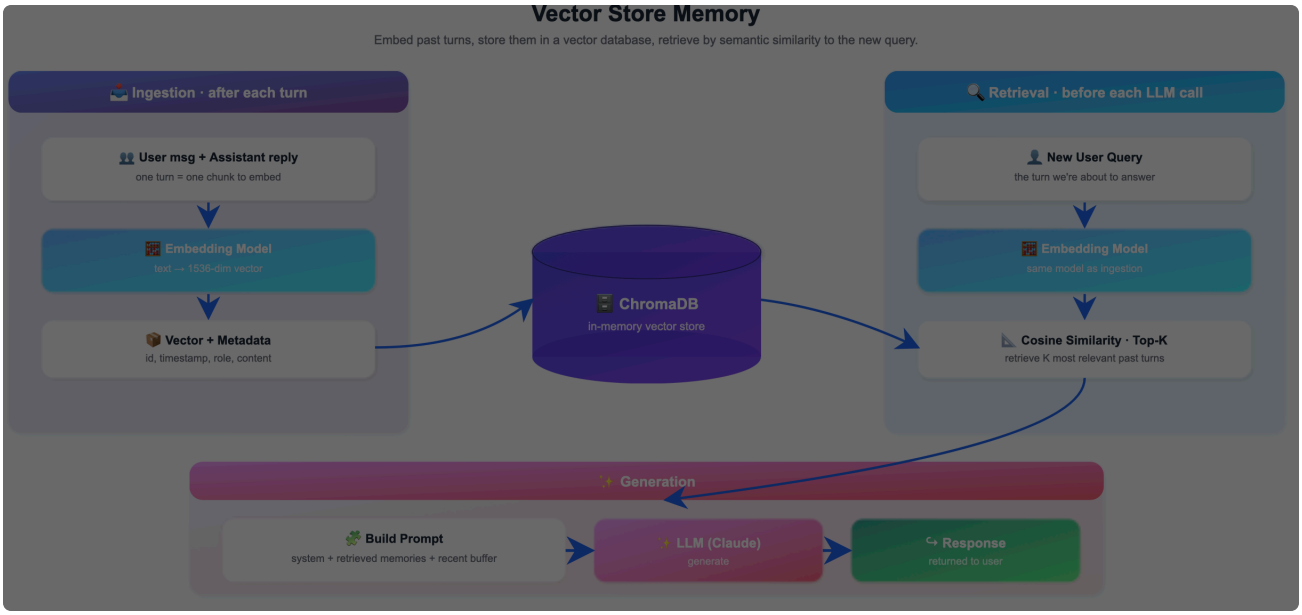
Verdict

The most predictable short-term memory. Weakest on recall — so only ship it with a long-term layer behind it.

PART B — LONG-TERM MEMORY

Everything above dies when the session ends. Everything below survives it.

6. Vector Store Memory



Core idea in one line

Turn every conversation turn into a vector, store it permanently, and retrieve the most **semantically relevant** past messages — not the most recent ones.

The analogy

Types 1–5 were a notepad on the advisor's desk; when it filled up, old pages fell off. This is a **searchable filing cabinet**. Every turn from every session ever is filed. When a new question arrives, the system searches the cabinet, pulls the relevant files, and puts them on the desk.

```
"Should I rebalance my portfolio?"
        ↓ embed the query
        ↓ search the vector DB
retrieved: "I'm conservative with investments"     (3 months ago)
           "I have ₹2L in equity mutual funds"     (2 sessions ago)
           "I'm buying a house in 2 years"         (last session)
        ↓ inject + current conversation
personalised, cross-session answer
```

Why does this exist?

Every short-term type retrieves by **when** something was said. This retrieves by **what it means**.

What is an embedding? A list of numbers representing meaning. Similar meanings sit close together:

```
"My salary is ₹1,20,000"      → [0.12, -0.45, 0.78, ...]
"I earn ₹1.2 lakh per month"  → [0.13, -0.44, 0.77, ...]  ← close
"I enjoy playing cricket"     → [0.89,  0.21, -0.45, ...] ← far
```

Search for "what is my income" and both salary sentences match, even though they share almost no words. That is **semantic search**.

When to use

- Returning users — the single most common trigger
- Conversations spanning weeks or months
- When the useful fact is old and unpredictable, so you cannot pre-select it
- Anywhere you already run a vector database

Advantages

- **True cross-session memory.** The first type in the list that survives a session ending.
- **Relevance beats recency.** A fact from six months ago surfaces if it matters now.
- **Scales indefinitely.** Storage is cheap; retrieval cost stays flat because you always pull top-K.
- **Cheap to run.** `text-embedding-3-small` costs roughly $0.02 per million tokens.

Disadvantages

- **No sense of time — this is the big one.** The store has no idea which of two contradicting memories is current. Chiru said "salary ₹1,20,000" in January and "salary ₹1,50,000" in June. Both are stored. Both match a salary query. The agent may confidently quote the old one. This is the **stale fact problem**, and it is the reason types 7 and 8 exist.
- **Retrieval latency.** An embedding call plus a search on every turn.
- **Retrieval can miss.** If the query is phrased oddly, the right memory may not surface — and the agent has no way to know it missed.
- **Multi-tenant risk.** Get the `user_id` filter wrong and one user sees another's financial data. Test this explicitly.

Worked example — FinCoach

Session 1 in March, Chiru shares his full profile. Session 2 in June, fresh session, buffer empty. He asks *"is my emergency fund big enough?"* — the vector store retrieves the March messages about expenses and the FD, and FinCoach answers as though it had been there all along.

Then the failure: he got a raise in April and mentioned it. Both salary figures are in the store. He asks about SIP capacity, retrieval returns the March salary, and the advice is built on a number that is three months out of date. **Mitigation is recency-weighted retrieval** — boost the similarity score by how recent the memory is — but that is a patch, not a cure. The cure is entity memory.

In a RAG project: this is where freshers most often get confused, so make the distinction sharp. **RAG retrieves from a document corpus. Vector store memory retrieves from conversation history.** Same technology — embeddings, cosine similarity, top-K — pointed at two different collections. A production RAG agent runs both: one index of documents, one index of what this user has said. Same code, two collections.

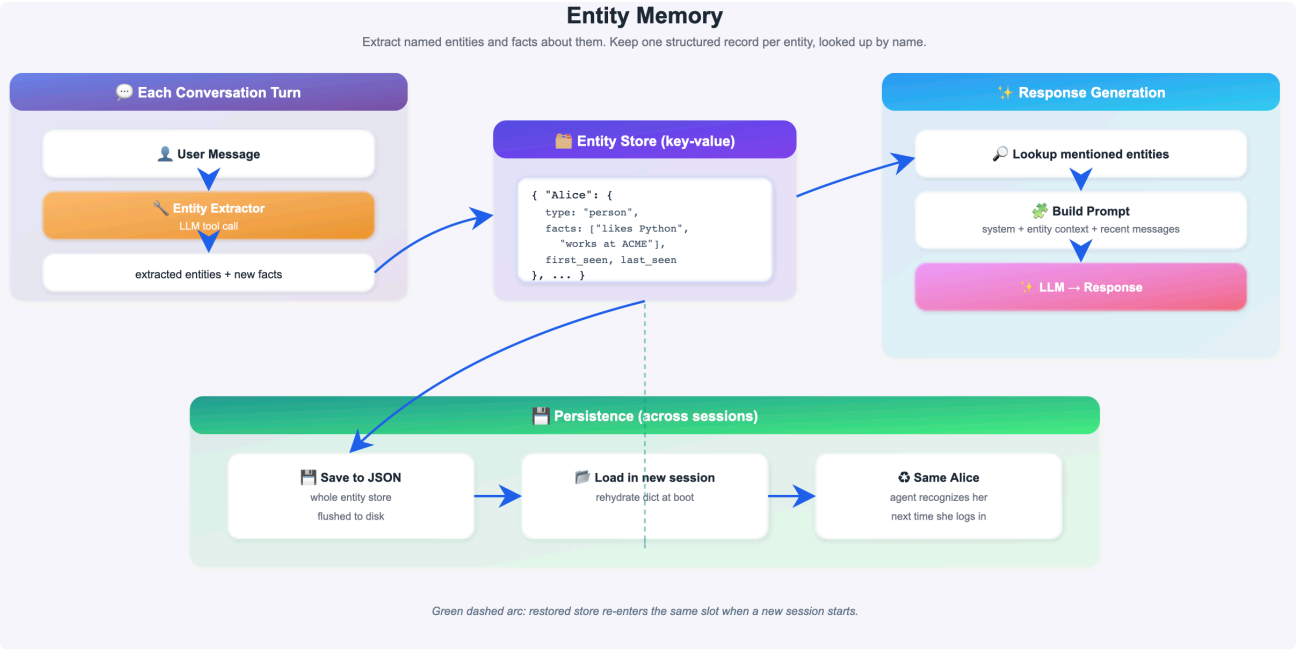
Hybrid note

The default long-term layer. Pair it with **Sliding Window (2)** or **Token Buffer (5)** for the current session. Evicted turns flow into the vector store; the vector store feeds relevant history back. And pair it with **Entity Memory (7)** to cover the stale-fact hole — vector store for fuzzy narrative recall, entity memory for current values.

Verdict

The workhorse of long-term memory. Excellent at recall, blind to time. Never let it own facts that change.

7. Entity Memory



Core idea in one line

Extract structured facts about named entities from the conversation, store them as a profile, and **update them in place** when new information arrives — so the profile always shows the current state.

The analogy

A CRM record that updates itself. When a customer says "I changed jobs," the agent updates the record — the old job is *replaced*, not appended. The record always reflects reality now.

```
ENTITY PROFILE — chiru_001
──────────────────────────────────────
personal:   name Chiru | age 32 | Mumbai
financial:  salary ₹1,20,000 | expenses ₹60,000
            surplus ₹60,000 | risk_profile conservative
assets:     fd_amount ₹50,000 | fd_maturity 3 months
goals:      retirement_age 55 | emergency_fund 6 months
──────────────────────────────────────
last_updated: 2026-06-12
```

Why does this exist?

To fix the stale fact problem. The difference from vector store is the **update model**:

	Vector Store (6)	**Entity Memory (7)**
Stores	Raw message text	Structured key-value facts
Retrieval	Similarity search	Direct lookup
Update	Append-only	**In-place — old value replaced**
Stale facts	Old and new both retrieved	Only the current value exists
Token cost	Variable	Fixed — the profile size

And a second key difference: **the profile is always injected, never retrieved.** No search, no similarity score, no chance of a miss. It is a small fixed block at the top of every call.

When to use

- Any product with facts that change: salary, address, job, portfolio, subscription tier
- When exact values matter and approximations are unacceptable
- Regulated domains where the agent must act on current data
- **Any financial, medical, or legal agent — effectively mandatory**

Advantages

- **Always current.** In-place update means contradictions cannot survive.
- **Fixed, small token cost.** Predictable and cheap.
- **Zero retrieval risk.** Always present, never missed.
- **Human-readable and auditable.** You can show a user their profile, and they can correct it.

Disadvantages

- **Extraction is imperfect.** The LLM must correctly identify and structure facts from messy speech. It will sometimes miss, sometimes hallucinate a field.
- **Entity disambiguation is genuinely hard.** Two people named Sharma, or "my HDFC account" when there are three, and the merge logic breaks.
- **Schema is a design decision you must make up front.** A field you did not anticipate cannot be stored. Rigid by nature.
- **Only holds what fits the schema.** Everything narrative or fuzzy falls outside it.
- **An extra LLM call per turn** for extraction.

Worked example — FinCoach

Chiru gets a raise: *"I've moved to ₹1,50,000 a month."* The extractor returns `{"monthly_salary": "₹1,50,000"}`. The profile field is **overwritten**. The old figure no longer exists anywhere in the profile. Every subsequent call sees ₹1,50,000 and only ₹1,50,000. Compare this with the vector store, where both numbers coexist forever and the agent has to guess.

In a RAG project: entity memory holds the user's operating context so retrieval can be filtered by it — their department, their region, their access tier, the specific product they own. Retrieval quality improves sharply when you can narrow the corpus using known facts about who is asking.

Hybrid note

Entity Memory and Vector Store are **complementary, not competing** — this is the most important pairing in this document.

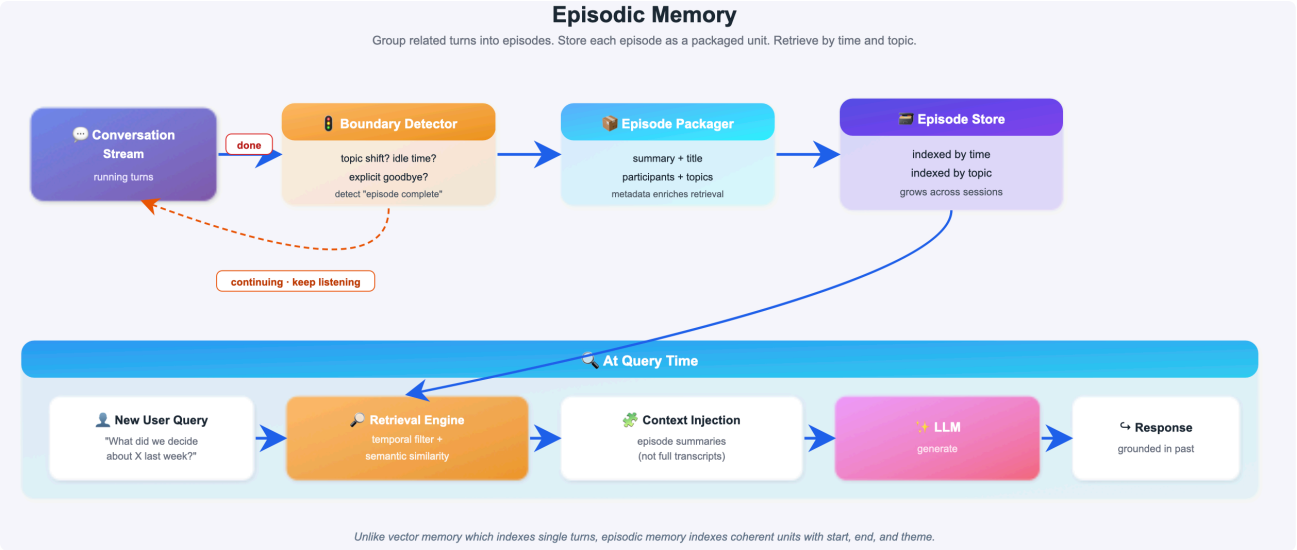
- **Entity memory** owns facts that change and must be exact. *What is true right now.*
- **Vector store** owns narrative, context, and open-ended recall. *What was discussed, whenever.*

Pair either with **Token Buffer (5)** or **Summary Buffer (4)** for the current session and you have a complete, production-viable architecture from three components.

Verdict

The correct home for any fact that changes. Pair with a vector store — neither is sufficient alone.

8. Episodic Memory :



Core idea in one line

Store every session as a complete, timestamped **episode** — what was discussed, what was decided, what advice was given — and retrieve whole episodes when they are relevant.

The analogy

A structured diary. Not a list of facts — a record of bounded experiences, each tied to a time.

```
EPISODE: session_vs_001
──────────────────────────────────────────────────
date:         2026-06-12        duration: 18 min
topics:       [salary, expenses, FD, SIP, retirement]
decisions:    Chiru will start a ₹5,000/month SIP in debt funds
advice_given: Recommended short-duration debt fund for FD proceeds
user_state:   Anxious about volatility; conservative stance reinforced
key_numbers:  {salary: ₹1,20,000, surplus: ₹60,000, FD: ₹50,000}
outcome:      Positive — user committed to the plan
──────────────────────────────────────────────────
```

Why does this exist?

Because some questions are about **events**, not facts. *"What did we decide last time?"* cannot be answered by a profile lookup or a similarity search over individual messages. It needs the session as a unit.

The cognitive-science framing is worth a minute in class. Tulving (1972) split long-term memory into **episodic** — specific events, what happened when — and **semantic** — general knowledge, detached from when it was learned. Types 8 and 9 in this repo are exactly that split.

The key property: **episodes are immutable.** Entity memory overwrites. Episodes never change. What happened on June 12th happened, permanently.

When to use

- Multi-session products where users expect continuity
- Coaching, therapy, project management, advisory — anything that tracks progress over time
- When users ask "when did we…" or "what happened during…"
- **When you need an audit trail** — this is your compliance layer

Advantages

- **Answers temporal questions** no other type can.
- **Immutable — a real audit record.** In regulated industries, this alone justifies the type.
- **Coherent, not fragmented.** An episode carries the reasoning and outcome, not isolated sentences.
- **Enables case-based reasoning.** Retrieve past episodes with good outcomes and use them to guide current advice.

Disadvantages

- **Episode boundary detection is the hard problem.** Where does one episode end? Session-based is easy but crude — one session may cover four unrelated topics. Topic-based is better and much harder.
- **Summaries lose detail.** Specific numbers can vanish from the episode summary, same as type 3.
- **Retrieval latency.** Embedding plus episode scan on every turn.
- **Unbounded storage growth.** Every session adds an episode forever. Needs type 13.
- **Only useful after several sessions.** Session 1 has no episodes — the value builds slowly.

Worked example — FinCoach

Chiru returns after three months: *"What did we decide about my FD last time?"* The episodic store retrieves the June episode and FinCoach answers precisely: the decision, the fund recommended, the reasoning, and the fact that he seemed anxious about volatility at the time. No other memory type can produce that answer.

In a RAG project: episodes record what a user investigated and concluded in each research session. When they return, the agent can open with "last time you were comparing vendor A and vendor B and leaned toward A on pricing — want to continue?" instead of starting cold.

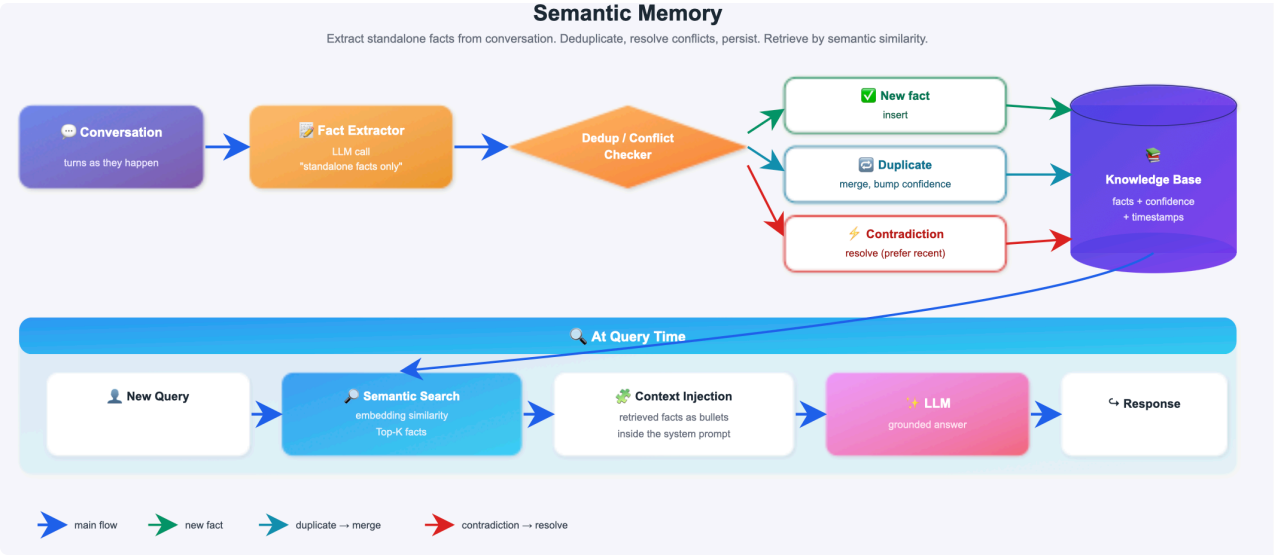
Hybrid note

Episodic memory is the **raw material for semantic memory (9)** — episodes go in, distilled patterns come out. It is also the natural pairing with **Entity Memory (7)**: entity holds current state, episodic holds how that state came to be. In a full stack: short-term for now, entity for facts, episodic for history, semantic for patterns.

Verdict

The memory of experience. Non-negotiable if your product spans sessions or needs an audit trail.

9 . Semantic Memory :



Core idea in one line

Distil general, durable truths from raw episodes — strip away the "when" and "where" and keep only what is always true.

The analogy

A distillery. Episodes are the raw ingredients. Semantic facts are the concentrated essence.

```
RAW EPISODES:
  Session 1: asked about FD options, seemed anxious, chose lowest risk
  Session 3: market dip mentioned — immediately asked to move to FD
  Session 5: hesitated on equity SIP, cited fear of loss, declined
  Session 7: asked about debt fund safety in a rising-rate environment
                        ↓ DISTILLATION ↓
SEMANTIC FACTS:
  "Chiru has strong loss aversion — prioritises capital preservation"
  "Market volatility triggers anxiety — needs reassurance before
deciding"
  "De facto risk profile is conservative regardless of stated goals"
```

Why does this exist?

You know Paris is the capital of France, but you cannot recall the moment you learned it. The fact outlived the episode. Agents need the same: a growing profile of durable truths that no longer depends on any single conversation.

The crucial distinction from entity memory (7) — teach these side by side, because freshers mix them up constantly:

	Entity Memory (7)	**Semantic Memory (9)**
Format	Structured key-value	Free-form natural language
Source	What the user **stated**	What the agent **observed across sessions**
Example	`risk_profile = conservative`	"Panics during volatility despite calling himself balanced"
Nature	Precise, schema-bound	Fuzzy, behavioural, inferred

Entity memory records what the user said about themselves. Semantic memory records what is true about them whether or not they said it. Facts here carry a **confidence score and an observation count**, and they can strengthen, weaken, or be contradicted over time.

When to use

- Long-lived relationships — months or years with the same user
- When behaviour matters more than stated preference
- Personalisation that should feel earned rather than configured
- When you want the agent to feel *experienced*, not just informed

Advantages

- **Cheapest long-term layer per token.** Compact fact statements, high information density.
- **Captures what users will not tell you.** Nobody says "I panic during market dips." The pattern reveals it.
- **Improves with time.** More sessions, better facts.
- **Facts evolve.** Contradictions weaken a fact rather than silently coexisting with it.

Disadvantages

- **Inference can be wrong, and confidently so.** Three cautious decisions might reflect three cautious market weeks, not a personality trait.
- **Needs volume.** Below roughly five sessions there is no pattern to distil.
- **Uncomfortable when surfaced badly.** "You tend to panic" is accurate and alienating. Design carefully what the agent says out loud.
- **Distillation is an expensive batch job.**
- **Serious privacy weight.** You are storing inferred psychological characteristics. Know your regulatory position before you ship this.

Worked example — FinCoach

Across seven sessions, FinCoach distils: *"Chiru consistently deflects equity discussions."* On session eight, before Chiru says a word, the agent already leads with debt instruments and frames everything in terms of capital safety. He experiences an advisor who finally gets him. No single session contained that fact.

In a RAG project: semantic facts tune retrieval and presentation. "This user prefers technical depth over summaries" changes which chunks you surface and how you write the answer — learned from behaviour, never asked.

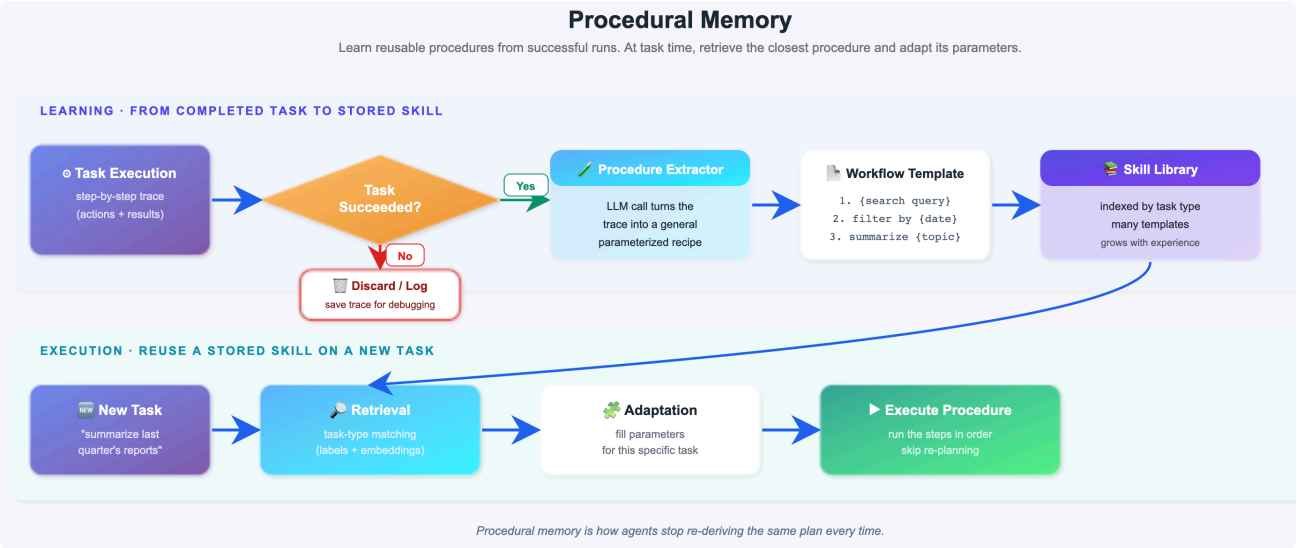
Hybrid note

Semantic memory is **downstream of episodic (8)** — episodes are its input, so you cannot run this without that. It sits alongside **Entity (7)**, never replacing it: entity for stated facts, semantic for observed patterns. Both are injected; together they are still small.

Verdict

What makes an agent feel experienced. Requires episodic memory upstream and real care about privacy.

10. Procedural Memory



Core idea in one line

Store **how-to knowledge** — the workflows, decision rules and interaction patterns that have proven effective — and inject them as operating instructions that shape every response.

The analogy

A skill manual that writes itself. You do not recall the day you learned to ride a bicycle; you just know how. The knowledge lives in execution, not recall.

```
FINCOACH PROCEDURAL MEMORY
═══════════════════════════════════════════════════

[WORKFLOW] Investment consultation, conservative users
  1. Confirm risk profile before any recommendation
  2. Lead with capital safety, then returns
  3. Present exactly 3 options, ranked by risk
  4. Quantify worst case before the user asks
  5. End with a numbered action list
  Learned from: 6 sessions, consistently positive outcomes

[RULE] Equity deflection handling
  When the user resists equity: acknowledge, validate the
  conservative stance, redirect to debt instruments.
  Never push equity twice in one session.
  Learned from: 4 sessions where pushing caused frustration
═══════════════════════════════════════════════════
```

Why does this exist?

Types 6–9 store *what the agent knows about the user*. This stores *how the agent should act*. It is the third pillar of Tulving's split: episodic (what happened), semantic (what is true), procedural (how to do it).

Two mechanics matter:

- **It lives in the system prompt.** Procedures are not retrieved and cited — they are injected as instructions and become part of how the agent behaves.
- **Procedures are learned from outcomes.** A workflow that produced good results gets reinforced; one that caused frustration gets weakened. Confidence scores go up and down based on what actually happened.

This is the bridge to self-improving agents. The landmark references are **Voyager** and **Agent Workflow Memory**.

When to use

- Agents that repeat similar multi-step tasks
- When users state hard constraints that must never be violated
- When you have discovered an interaction sequence that works and want it applied consistently
- Products where consistency of *approach* matters as much as correctness of *facts*

Advantages

- **Consistency across sessions.** The agent handles the same situation the same way every time.
- **Hard constraints are enforced structurally.** "Never recommend equity to me" becomes a rule, not a hope.
- **Improves without retraining.** Learning lives in natural language, not weights.
- **You can seed it.** Do not wait for procedures to be learned — pre-load your domain expertise on day one. This is the fastest path to value in this whole document.

Disadvantages

- **Procedures may not generalise.** What worked for a conservative user may be wrong for an aggressive one. Scope them, or they misfire.
- **The library needs curation as it grows.** Twenty procedures in the system prompt start conflicting with each other.
- **System prompt bloat.** Every procedure costs tokens on every single call, forever.
- **Bad procedures entrench.** Learn the wrong lesson and the agent repeats it confidently until someone notices.

Worked example — FinCoach

Chiru says *"never recommend equity to me again."* That becomes a procedural rule, not a stored message. Six months later, in a different session with a completely fresh context, the rule is in the system prompt and the agent simply does not raise equity. Compare with a vector store, where that instruction is one message among thousands and may or may not be retrieved.

In a RAG project: procedures govern how retrieved material is used — "always cite the source document and section," "if confidence is low, say so before answering," "never answer from a document older than the current policy version." Behaviour rules, not knowledge.

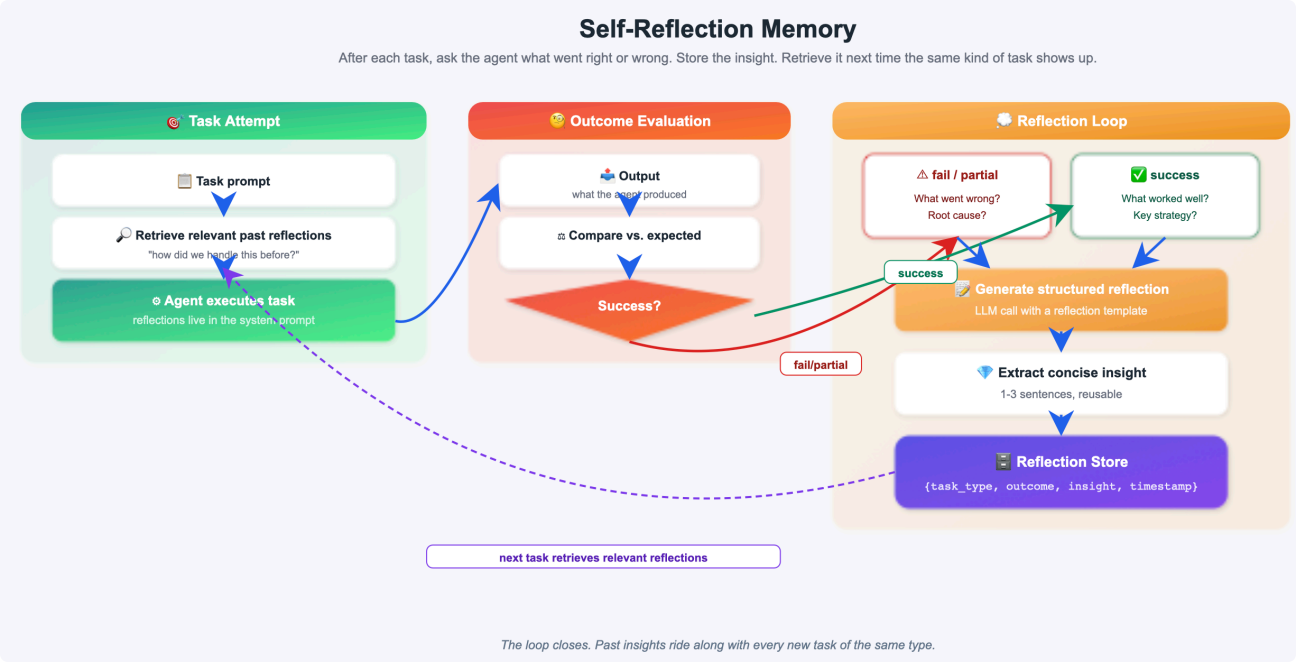
Hybrid note

Procedural memory pairs with **Self-Reflection (11)**, which supplies it with better procedures. And with **Semantic (9)**: semantic memory says *what kind of user this is*, procedural says *how to handle that kind of user*. Together they produce personalisation that is both accurate and consistent.

Verdict

The memory of skill. Seed it with your domain expertise from day one — do not wait for it to learn.

11 . Self - Reflection Memory



Core idea in one line

After each session, the agent reviews its own responses — what it did well, what it did badly, what it should do differently — and writes structured improvement notes that feed into future sessions.

The analogy

A chess player reviewing their games. Not just recording the moves — asking *"why did I lose that piece? What pattern did I miss?"* Over time the post-game notes become more valuable than the game records.

Why does this exist?

Every other memory type records *what happened*. This records *why it went well or badly, and what to do differently*. The research foundation is **Reflexion (Shinn et al., 2023)**, which showed that agents able to reflect on failed attempts and carry those reflections forward substantially outperformed agents that simply retried.

The loop has four steps:

1. The agent attempts a task and gets an outcome (success, partial, failure)
2. A reflection prompt asks it to analyse root causes and extract lessons
3. The insight is stored in a dedicated reflection memory
4. Before future tasks, relevant reflections are retrieved and injected

The agent improves **without any parameter updates**. Learning lives entirely in natural-language notes.

Compare with procedural memory — freshers will ask, and the distinction is subtle:

	Procedural (10)	**Self-Reflection (11)**
Source	Patterns extracted from session content	The agent's critique of its own performance
Perspective	Third-party observation	First-person self-assessment
Content	"How to handle X"	"I should have done Y instead of Z"

Procedural learns what to do. Self-reflection learns from mistakes.

When to use

- Agents that repeat similar tasks and should get better at them
- When outcomes are measurable — did the user act on the advice or not?
- High-stakes domains where consistency errors are costly

- Research and agentic workflows with clear success and failure signals

Advantages

- **Genuine improvement over time**, without fine-tuning.
- **Catches errors nothing else catches** — contradictions with past advice, missed angles, violated constraints.
- **Human-readable.** You can audit exactly what the agent thinks it learned.
- **Cheap relative to retraining.** One extra call per session.

Disadvantages

- **Quality depends entirely on the model's reasoning.** A weak model writes weak reflections.
- **Hallucinated self-critique is the main failure mode.** The agent invents a mistake it never made and then "corrects" for it — degrading behaviour based on a fiction. Ground reflections in actual outcome signals, not vibes.
- **Extra LLM call per session.**
- **Reflections accumulate and eventually conflict.** Needs curation and pruning.
- **Requires outcome signals to be useful.** Without knowing whether the advice worked, reflection is guesswork.

Worked example — FinCoach

End of session: the agent reviews itself against five checks — consistency (did I contradict past advice?), completeness (did I miss an angle?), constraint compliance (did I respect "never equity"?), clarity, and outcome. It notes: *"I recommended a debt fund without first confirming the FD maturity date. Next time, confirm timelines before recommending any instrument."* That note is injected into the next session. The mistake does not recur.

In a RAG project: the agent reflects on retrieval quality — *"the user reformulated their question twice, meaning my first retrieval missed. For queries about pricing, search the contracts index rather than the product index."* That lesson improves retrieval on the next run.

Hybrid note

Self-reflection **feeds procedural memory (10)**. Reflections are raw lessons; validated, repeated lessons get promoted into procedures. Reflections that keep proving true become rules. This is the top of the memory stack and the most advanced of the eleven — do not attempt it before the layers beneath it are stable.

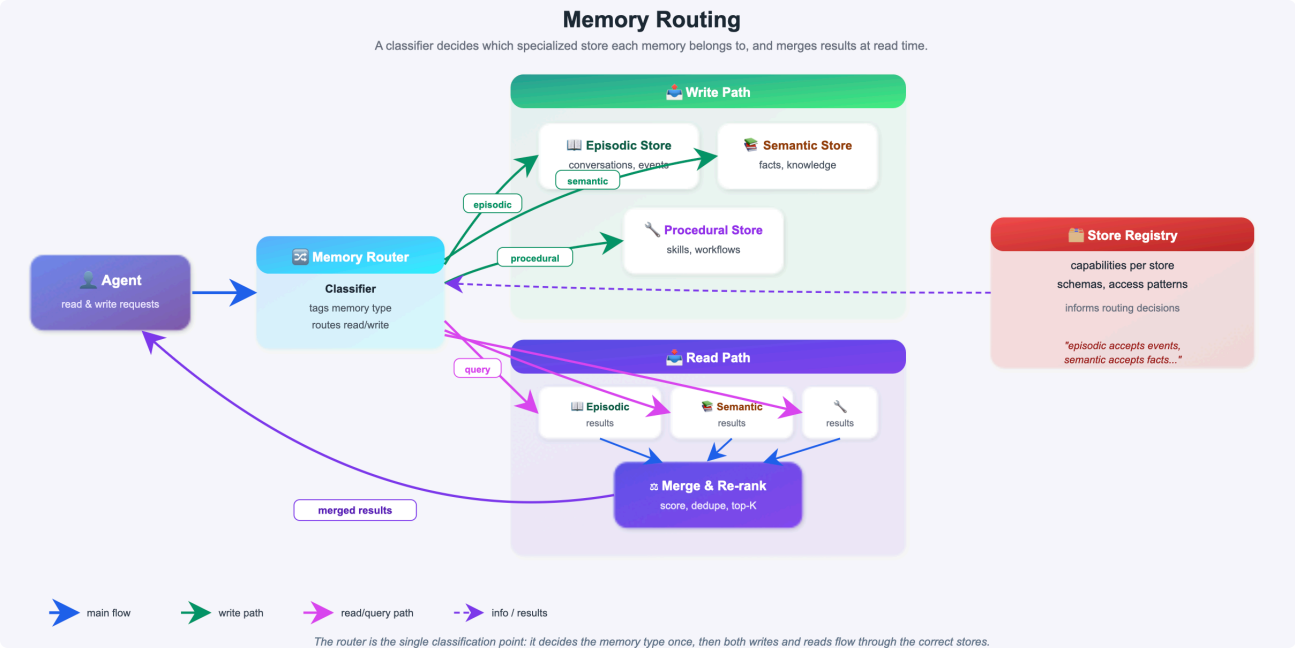
Verdict

The memory of learning. The most advanced type here. Build it last, and ground it in real outcomes.

PART C — ORCHESTRATION AND HYGIENE

You now have six long-term stores. These last two types manage them.

12. Memory Routing



Core idea in one line

Analyse each incoming message for content type and intent, then dispatch it to the **correct** memory stores — rather than blindly querying and writing to all of them every turn.

The analogy

An air traffic controller. A plane arrives; the controller does not send it to every runway at once. They look at the type, size and destination, and route it to exactly one gate.

```
"What is my current salary?"            → ENTITY (current fact)
"What did we decide last April?"        → EPISODIC (past session)
"How does a SIP work?"                  → VECTOR (general retrieval)
"I just changed jobs to TCS"            → ENTITY update + VECTOR write
"Never recommend equity to me again"    → PROCEDURAL (constraint write)
"I'm worried about market volatility"   → SEMANTIC (behaviour pattern)
```

Why does this exist?

Because once you have five or six stores, querying all of them on every turn is enormously wasteful.

Without routing: entity 150 + vector 200 + episodic 250 + semantic 200 + reflection 150 = **950 tokens injected on every turn**, most of it irrelevant.

With routing: *"How does a SIP work?"* touches the vector store only — **200 tokens**. *"What is my salary?"* touches entity only — **150 tokens**.

60–80% token reduction with no loss in answer quality.

Routing runs in two directions:

- **READ routing** — which stores to query *before* generating a response
- **WRITE routing** — which stores to update *after* new information arrives

When to use

- **Whenever you run three or more memory stores.** Below three, the classification overhead is not worth it.
- Cost-sensitive production systems
- When context window pressure is real

Advantages

- **Large, immediate cost reduction.** The clearest ROI of any technique here.
- **Better answers, not just cheaper ones.** Less irrelevant context means less distraction for the model.
- **Faster.** Fewer store queries per turn.
- **Scales.** Adding a seventh store does not add cost to every turn.

Disadvantages

- **A classification call on every message — typically 200–500ms.** You are trading store-query cost for classification latency.
- **Misrouting fails silently.** Route a salary update to the vector store instead of entity memory and nothing errors — the fact is simply in the wrong place and the profile is now stale. This is the dangerous failure mode.
- **Routing rules need maintenance** as stores are added.
- **Ambiguous messages route badly.** "Tell me about my investments" could legitimately hit three stores.

Worked example — FinCoach

Chiru: *"I just moved to TCS and my salary is now ₹1,50,000."* The router classifies this as a fact update and dispatches to **entity** (overwrite salary and employer) **and vector** (store the raw message for future narrative recall). It does **not** touch episodic, semantic, procedural, or reflection. Two writes instead of six, and each fact lands where it belongs.

In a RAG project: the same idea splits document retrieval from memory retrieval. "What is our refund policy?" goes to the document index. "What did I ask you yesterday?" goes to conversation memory. "Does the refund policy apply to my plan?" goes to both. Routing prevents you from searching everything on every question.

Hybrid note

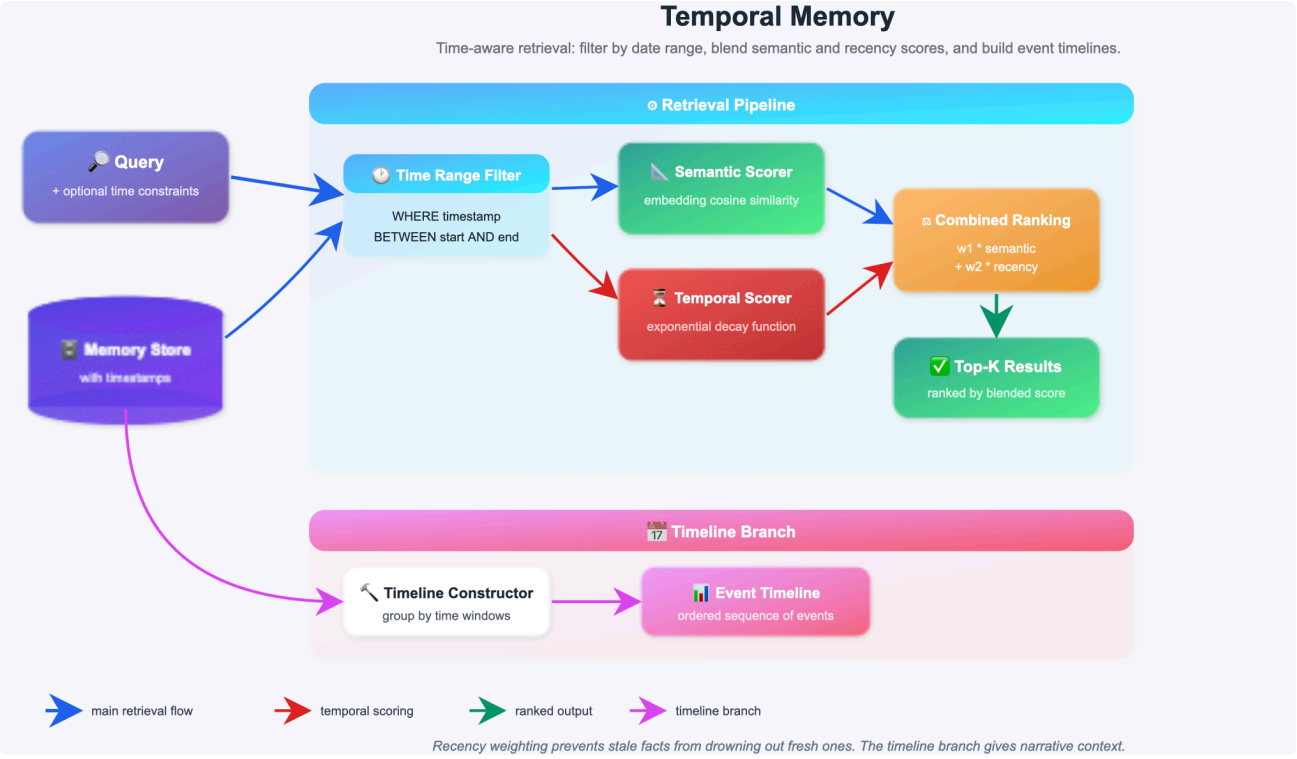
This is not a memory type — it is the **layer that makes hybrids affordable**. Without routing, a six-store architecture is unaffordable at scale. With it, adding stores is nearly free. Build routing at the point where you add your third store.

Safety rule to teach: for genuinely ambiguous messages, route to more stores rather than fewer. Over-fetching costs tokens; misrouting a write corrupts your data.

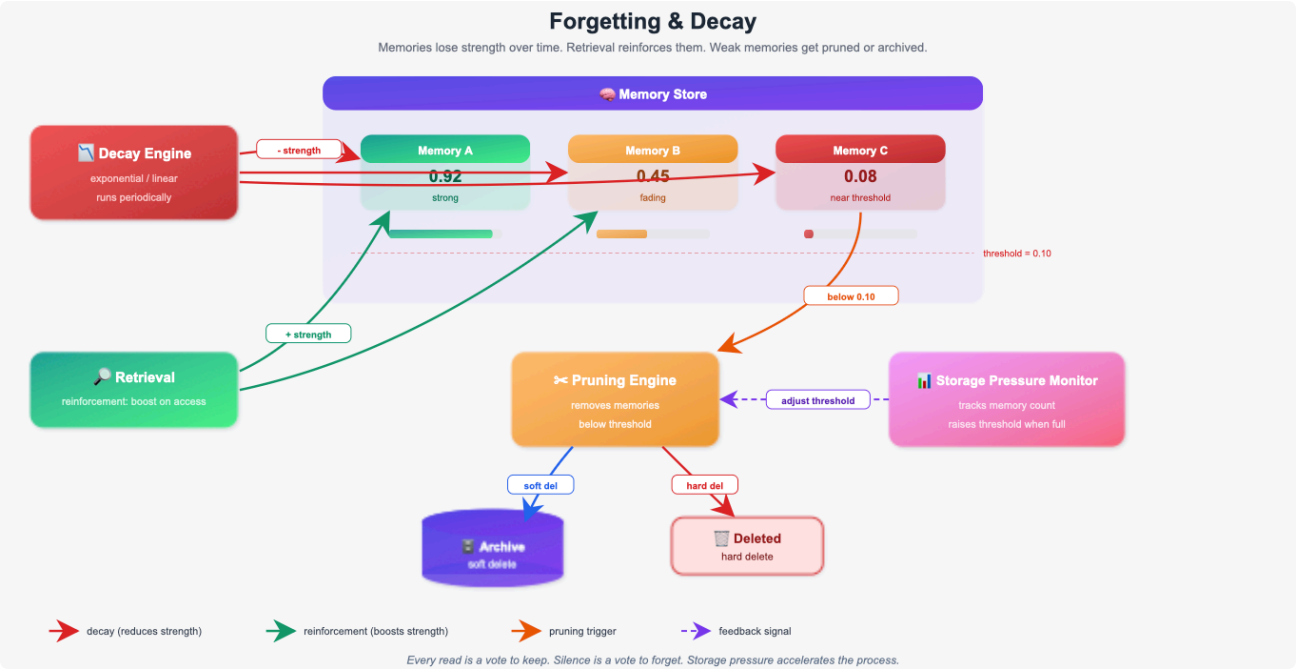
Verdict

Not memory — traffic control. Introduce it at your third store, and log every routing decision so you can catch misroutes.

13. Temporal Memory



14. Forgetting and Decay pattern



Core idea in one line

Deliberately remove or suppress memories that are no longer useful — based on **age**, **access frequency**, **importance**, or **budget** — so the store stays lean, fast and high-signal.

The counterintuitive framing

More memory is not better. An agent that remembers everything becomes slower to retrieve from, noisier in its answers, and unable to distinguish what matters now from what mattered two years ago. **Forgetting on purpose** is what keeps a long-running memory store trustworthy.

Why does this exist?

Every long-term type grows without limit. After a year of daily use, one user's store might hold 50,000 memories — most of them dead weight. Three consequences follow: retrieval slows, top-K fills with near-duplicates and stale facts, and storage costs climb.

The four common signals:

Signal	Rule	Best for
Age	Older than N months → archive	Time-sensitive facts
Access	Never retrieved in N queries → prune	Low-value noise
Importance	Below a score threshold → prune	Small talk
Budget	Store exceeds N memories → evict the weakest	Hard cost caps

Distinguish this from *temporal weighting*, which scores old memories lower but keeps them. Forgetting actually **removes or archives** them. Weighting bounds relevance; forgetting bounds size.

When to use

- Any long-running agent — after roughly six months of real usage, this stops being optional
- When retrieval quality is visibly degrading
- When storage cost is showing up on the bill
- **When regulation requires deletion** — GDPR, data retention policy, user right-to-erasure

Advantages

- **Retrieval quality goes up.** Removing stale memories directly improves what top-K returns.
- **Bounded, predictable storage cost.**
- **Faster retrieval.** Smaller index, quicker search.
- **Compliance.** For a fintech product, deletion capability is a legal requirement, not a feature.

Disadvantages

- **Deletion is irreversible.** Prune the wrong memory and it is gone. **Always archive before deleting** — move to cold storage rather than dropping.
- **Importance is hard to score.** A memory retrieved zero times may still be critical the one time it is needed.
- **Recency bias.** Aggressive age-based decay makes the agent forget genuinely durable facts alongside stale ones.
- **Tuning is empirical.** No universal thresholds — you have to measure on your own traffic.

Worked example — FinCoach

Chiru's store after two years: 12,000 memories. Small talk from March 2024 is retrieved zero times and scores near-zero on importance — pruned. His stated retirement goal is retrieved rarely but scores high on importance — **kept**. His January salary figure is superseded by the entity profile — archived, not deleted, so the audit trail survives.

The rule to write on the board: *never let access frequency alone decide.* Importance overrides frequency. A rarely-needed fact can be the most consequential one in the store.

In a RAG project: applies to both indexes. Document chunks from a superseded policy version should be archived so they stop competing in retrieval; conversation memories about a project that shipped last year should decay out. Stale retrieval is worse than no retrieval, because the agent answers confidently from outdated ground truth.

Hybrid note

Forgetting operates **across every long-term store**, with different policies per store:

Store	Policy
Vector (6)	Age + access decay — the highest-volume store, prune hardest
Entity (7)	Never prune fields; the in-place update *is* the forgetting
Episodic (8)	Archive old episodes to cold storage; never hard-delete (audit)
Semantic (9)	Prune facts whose confidence has decayed below threshold
Procedural (10)	Prune procedures with poor outcome history
Reflection (11)	Prune superseded lessons

Verdict

Deliberate forgetting is what keeps long-running memory usable. Archive, never hard-delete. Add it before your first anniversary in production.

PART D — CHOOSING AND COMBINING

The three questions that pick your memory type

Give participants this decision path. It replaces memorising thirteen things.

Q1 — Does the user come back?

- No, single session only → short-term only (types 1–5). Stop here.
- Yes → continue.

Q2 — What kind of thing must survive between sessions?

What must survive	Use
Facts that change (salary, address, plan)	**Entity (7)**
What was said, whenever	**Vector Store (6)**
What happened in a specific session	**Episodic (8)**

What must survive	Use
Patterns in how the user behaves	**Semantic (9)**
How the agent should act	**Procedural (10)**
What the agent got wrong	**Self-Reflection (11)**

Q3 — What is your binding constraint in the session?

Constraint	Short-term choice
Latency above all	**Token Buffer (5)**
Recall within the session	**Summary Buffer (4)**
Simplicity	**Sliding Window (2)**
Conversation is short anyway	**Buffer (1)**

Hybrid combinations that actually get built

Level 1 — Minimum viable memory

Token Buffer (5) + Vector Store (6)

The current session is capped at a fixed token budget; everything evicted flows into the vector store asynchronously and is searched back when relevant. No latency spikes, true cross-session recall, two components.

Weakness: the stale fact problem. The vector store has no sense of time, so contradicting facts coexist. Acceptable for content and support agents; **not acceptable where numbers matter**.

Level 2 — Production standard

Token Buffer (5) or Summary Buffer (4) + Entity (7) + Vector Store (6)

Adding entity memory closes the stale-fact hole. Facts that change live in a profile that is always injected and updated in place. The vector store handles narrative recall. This is the architecture most production agents should target.

How to choose the short-term half: token buffer when latency is the constraint, summary buffer when in-session recall is.

Level 3 — Relationship agent

Summary Buffer (4) + Entity (7) + Vector (6) + Episodic (8) + Semantic (9) + Routing (12)

Now the agent remembers sessions as events and has distilled patterns about who the user is. Routing becomes mandatory here — without it you would inject ~950 tokens of memory on every turn.

Level 4 — Self-improving agent

Level 3 + Procedural (10) + Self-Reflection (11) + Forgetting (13)

The full stack. The agent learns how to behave and learns from its own mistakes, and forgetting keeps the whole thing from collapsing under its own weight.

The one-page pairing rule

> **Short-term answers "what did we just say?" Long-term answers "who is this person?"**
>
> Always run one of each. Which short-term type you pick is a **latency and cost** decision. Which long-term types you pick is a **what-kind-of-question** decision.
>
> And one rule above all others: **never let a summary own a number.** Numbers belong in Entity Memory. Summaries drift; profiles do not.

Common misconceptions to correct in the session

"RAG is memory." No. RAG retrieves from documents; memory retrieves from conversation. Same technology, different collection. A real agent runs both.

"Bigger context windows make memory obsolete." No. A 1M-token window does not solve cost (you pay for every token), relevance (more context means more distraction), or persistence (the window still empties when the session ends).

"More memory is better." No — see type 13. More memory means slower retrieval and noisier answers.

"Long-term memory means saving the chat log." No. That is just a bigger buffer, and it carries the full cost back with it. Long-term memory means retrieving *what matters*, not

everything.

"Entity and semantic memory are the same thing." No. Entity stores what the user **stated** (`risk_profile = conservative`). Semantic stores what the agent **observed** ("panics during dips despite calling himself balanced"). Stated versus observed.